

Análisis sintáctico de un JSON simplificado

1. Introducción

El formato **JSON (JavaScript Object Notation)** es uno de los mecanismos más utilizados actualmente para el intercambio de información entre sistemas informáticos. Es ampliamente utilizado en aplicaciones web, servicios REST, microservicios y sistemas distribuidos.

JSON permite representar información estructurada mediante pares *clave-valor*. Cada clave es una cadena que identifica un atributo, mientras que el valor puede ser un número, una cadena u otro objeto JSON.

Por ejemplo, un objeto JSON simple puede representarse de la siguiente manera:

```
{ "nombre": "Juan" }
```

También es posible tener varios pares clave-valor dentro de un mismo objeto:

```
{ "nombre": "Juan", "edad": 20 }
```

Otra característica importante de JSON es que permite estructuras jerárquicas mediante objetos anidados:

```
{ "curso": { "nombre": "TC" } }
```

En este caso, el valor asociado a la clave **curso** es otro objeto JSON.

Desde el punto de vista de los lenguajes formales, la estructura de un documento JSON puede describirse mediante una **gramática libre de contexto**. Por esta razón, es posible construir analizadores sintácticos que verifiquen si un documento JSON cumple con una estructura válida.

En esta práctica se utilizará un **analizador léxico** (lexer) previamente proporcionado que identifica los tokens básicos del lenguaje. A partir de estos tokens, se implementará un **analizador sintáctico** (parser) que verificará si la secuencia de tokens corresponde a una estructura válida de acuerdo con una gramática definida.

El objetivo de la práctica es comprender cómo una **gramática libre de contexto** puede implementarse directamente mediante un parser descendente recursivo.

2. Análisis léxico y sintáctico

Para analizar la estructura de un archivo JSON es necesario realizar dos etapas principales del análisis de lenguajes: el **análisis léxico** y el **análisis sintáctico**.

El **analizador léxico** (lexer) tiene como objetivo recorrer el texto del archivo y dividirlo en unidades básicas llamadas *tokens*. Cada token representa un elemento significativo del lenguaje, como llaves, comas, cadenas o números.

Por ejemplo, para el siguiente JSON:

```
{ "nombre": "Juan" }
```

el lexer produce una secuencia de tokens similar a la siguiente:

```
(LLAVE_IZQ, {)
(CADENA, "nombre")
(DOS_PUNTOS, :)
(CADENA, "Juan")
(LLAVE_DER, })
```

El lexer que se proporciona en esta práctica utiliza expresiones regulares para identificar estos tokens. Un fragmento simplificado del código es el siguiente:

```
1 patrones_token = [
2     ("LLAVE_IZQ", r"\{"),
3     ("LLAVE_DER", r"\}"),
4     ("DOS_PUNTOS", r":"),
5     ("COMA", r","),
6     ("NUMERO", r"\d+"),
7     ("CADENA", r'"[~]*"')
8 ]
```

Cada patrón permite identificar un tipo de token dentro del texto del JSON.

Posteriormente, la función principal del lexer recorre el texto y genera la lista de tokens:

```
1 def obtener_tokens(texto):
2     tokens = []
3
4     for coincidencia in re.finditer(patron_general, texto):
5         tipo = coincidencia.lastgroup
6         valor = coincidencia.group()
7
8         if tipo != "ESPACIOS":
9             tokens.append((tipo, valor))
10
11     return tokens
```

El resultado de esta función es una lista de tokens que será utilizada por el analizador sintáctico.

2.1. Gramática inicial

El **analizador sintáctico** (parser) recibe la lista de tokens generada por el lexer y verifica si la secuencia corresponde a una estructura válida del lenguaje.

El parser inicial reconoce únicamente objetos JSON muy simples formados por un solo par clave-valor.

La gramática inicial es la siguiente:

$$\begin{aligned}OBJETO &\rightarrow \{PAR\} \\ PAR &\rightarrow CADENA : VALOR \\ VALOR &\rightarrow CADENA \mid NUMERO\end{aligned}$$

Esta gramática permite reconocer estructuras como:

```
{"nombre": "Juan"}  
{"edad": 20}
```

2.2. Implementación de la gramática en el parser

El parser proporcionado sigue el enfoque de un **parser descendente recursivo**. En este tipo de parser, cada **símbolo no terminal** de la gramática se implementa como una función del programa.

Por ejemplo, la regla

$$OBJETO \rightarrow \{PAR\}$$

se implementa en el parser mediante una función similar a la siguiente:

```
1 def analizar_objeto():  
2     consumir("LLAVE_IZQ")  
3     analizar_par()  
4     consumir("LLAVE_DER")
```

La producción

$$PAR \rightarrow CADENA : VALOR$$

se implementa mediante la función:

```
1 def analizar_par():  
2     consumir("CADENA")  
3     consumir("DOS_PUNTOS")  
4     analizar_valor()
```

Finalmente, la producción

$$VALOR \rightarrow CADENA \mid NUMERO$$

se implementa mediante una función que verifica el tipo de token actual:

```

1 def analizar_valor():
2
3     if token_actual()[0] == "CADENA":
4         consumir("CADENA")
5
6     elif token_actual()[0] == "NUMERO":
7         consumir("NUMERO")
8
9     else:
10        raise Exception("Valor no valido")

```

De esta manera se puede observar claramente que **cada símbolo no terminal de la gramática se convierte en una función del parser**. El analizador sintáctico recorre los tokens generados por el lexer verificando que la estructura del JSON cumpla con las reglas definidas por la gramática.

3. Ejecución del proyecto

En esta práctica se proporcionan dos programas en Python:

- `lexer_json.py`: implementa el analizador léxico.
- `parser_json.py`: implementa el analizador sintáctico.

El lexer se encarga de leer el archivo JSON y generar una lista de *tokens*. Posteriormente, el parser utiliza esta lista de tokens para verificar si la estructura del JSON es válida según la gramática definida.

3.1. Requisitos

Para ejecutar esta práctica únicamente se requiere tener instalado **Python 3**.

No es necesario crear un ambiente virtual ni instalar bibliotecas externas, ya que el código utiliza únicamente módulos estándar del lenguaje Python.

3.2. Organización de archivos

Se recomienda organizar los archivos de la siguiente manera:

```

json-parser/
|-- lexer_json.py
|-- parser_json.py
|-- ejemplo1.json
|-- ejemplo2.json
|-- ejemplo3.json
'-- .gitignore

```

3.3. Creación del archivo .gitignore

Al ejecutar programas en Python, el intérprete genera automáticamente un directorio llamado `__pycache__`. Este directorio contiene archivos temporales compilados y no es necesario almacenarlo en un repositorio de Git.

Para evitar que este directorio sea agregado al repositorio, se debe crear un archivo llamado `.gitignore` con el siguiente contenido:

```
__pycache__/
```

De esta forma, Git ignorará estos archivos automáticamente.

3.4. Ejecución del lexer

Para ejecutar el analizador léxico se utiliza el siguiente comando:

```
python lexer_json.py ejemplo1.json
```

Este programa leerá el archivo JSON y mostrará en pantalla la lista de tokens generados.

3.5. Ejecución del parser

Para ejecutar el analizador sintáctico se utiliza el siguiente comando:

```
python parser_json.py ejemplo1.json
```

El parser primero ejecutará el lexer para obtener los tokens y posteriormente verificará si la estructura del archivo JSON es válida según la gramática implementada.

Si el archivo cumple con la gramática definida, el programa mostrará un mensaje indicando que el JSON es válido. En caso contrario, se mostrará un mensaje de error indicando el problema encontrado.

4. Ejercicio 1: múltiples pares clave–valor

El primer ejercicio consiste en extender el parser para permitir objetos que contengan más de un par clave–valor separados por comas. La nueva gramática queda definida de la siguiente forma:

$$\begin{aligned}OBJETO &\rightarrow \{PARES\} \\ PARES &\rightarrow PAR \mid PAR, PARES \\ PAR &\rightarrow CADENA : VALOR \\ VALOR &\rightarrow CADENA \mid NUMERO\end{aligned}$$

Con esta extensión, el parser deberá reconocer estructuras como las siguientes:

```
{"nombre":"Juan","edad":20}  
{"curso":"TC","creditos":10}
```

5. Ejercicio 2: objetos anidados

El segundo ejercicio consiste en extender la gramática para permitir que un valor también pueda ser otro objeto JSON. Esto introduce recursión en la gramática. La gramática resultante es la siguiente:

$$\begin{aligned}OBJETO &\rightarrow \{PARES\} \\ PARES &\rightarrow PAR \mid PAR, PARES \\ PAR &\rightarrow CADENA : VALOR \\ VALOR &\rightarrow CADENA \mid NUMERO \mid OBJETO\end{aligned}$$

Con esta extensión, el parser deberá reconocer estructuras anidadas como las siguientes:

```
{"curso":{"nombre":"TC"}}  
{"alumno":{"nombre":"Ana","edad":22}}
```

En este punto, el parser deberá ser capaz de analizar objetos que contienen otros objetos dentro de ellos, lo que representa una característica fundamental de las gramáticas libres de contexto: la posibilidad de definir estructuras recursivas.